PYTHON CASE STUDIES

Complete Project Report: 10 Problem-Solving Programs

Includes problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python source code, test cases, sample output screens, and conclusions.

Prepared for academic practice

Note: Thresholds, rates, fees, and policies are illustrative academic assumptions. Replace them with values supplied by your instructor or the relevant organization.

How to Use This Report

Each case study is self-contained and uses beginner-friendly Python constructs. The flowcharts are text-based vertical flow diagrams. The sample output sections are formatted output previews that you can reproduce by running the included code in IDLE, VS Code, or another Python interpreter.

Currency values use Indian rupees (■). All sample values are illustrative; the programs are educational examples, not official billing, medical, railway, tax, or financial systems.

Contents

1. University Scholarship Allocation

2. Progressive Electricity Billing

3. Hospital Billing and Patient Classification

4. Employee Payroll and Deduction Analysis

5. Multi-Plan Telecommunications Cost Optimization

6. Railway Fare and Reservation Calculator

7. Retail Billing with Multiple Discount Rules

8. Manufacturing Cost and Profitability Analysis

9. Multi-Currency Travel Budget Analysis

10. Campus Event Financial Planning

1. University Scholarship Allocation

Problem Statement

Develop a program that checks applicants against aggregate marks, minimum subject scores, attendance, and family-income criteria. It lists eligible applicants in merit order and resolves ties using higher aggregate, then higher attendance, then lower family income.

Objectives

• Validate applicant data and apply all eligibility rules consistently.

• Rank eligible applicants using predefined tie-break rules.

• Display eligible and ineligible applicants with reasons.

Assumptions

• Marks and attendance are percentages from 0 to 100; income is non-negative.

• Eligibility thresholds are aggregate ≥ 80, every subject ≥ 70, attendance ≥ 75, and annual family income ≤ ■300,000.

• Tie-break order: aggregate descending, attendance descending, income ascending.

Algorithm

1. Read applicant records and thresholds.

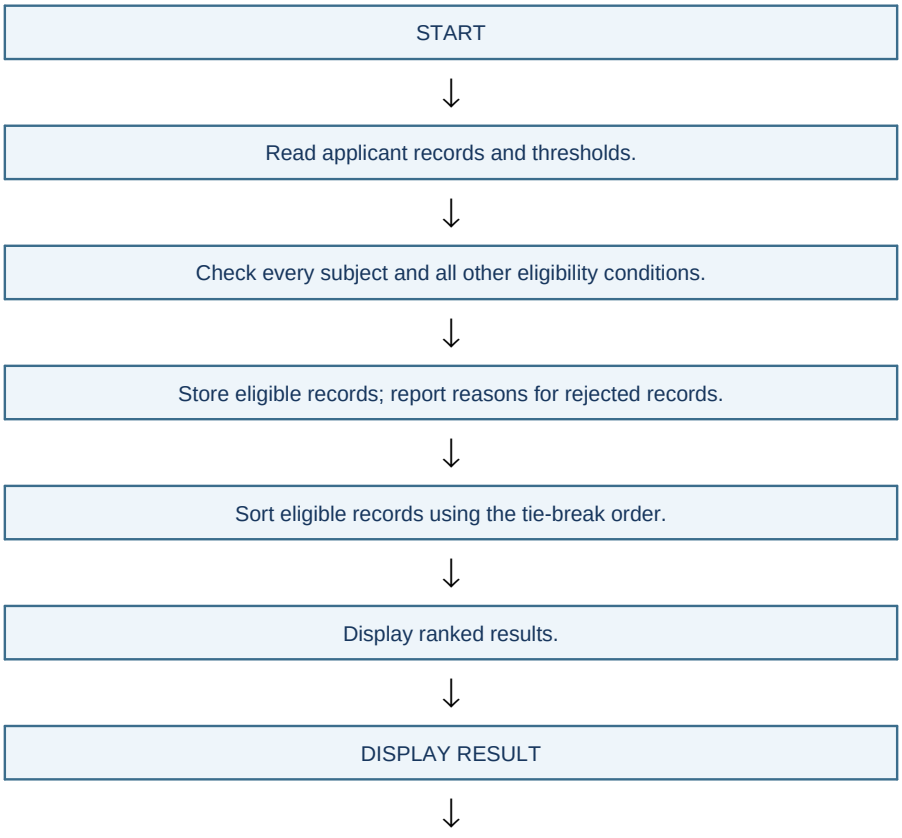
2. Check every subject and all other eligibility conditions.

3. Store eligible records; report reasons for rejected records.

4. Sort eligible records using the tie-break order.

5. Display ranked results.

Flowchart



STOP

Pseudocode

```
START\nREAD applicants\nFOR each applicant\n  CHECK aggregate, subject minimums, attendance, income\n  IF all checks pass: a
```

Python Source Code

```
applicants = [
    {"name": "Asha", "aggregate": 91, "subjects": [88, 92, 85],
     "attendance": 90, "income": 180000},
    {"name": "Bala", "aggregate": 84, "subjects": [76, 81, 72],
     "attendance": 82, "income": 250000},
    {"name": "Charan", "aggregate": 79, "subjects": [90, 85, 88],
     "attendance": 90, "income": 150000},
    {"name": "Divya", "aggregate": 88, "subjects": [90, 68, 92],
     "attendance": 95, "income": 120000},
]
eligible = []
for a in applicants:
    reasons = []
    if a["aggregate"] < 80: reasons.append("aggregate below 80")
    if min(a["subjects"]) < 70: reasons.append("subject below 70")
    if a["attendance"] < 75: reasons.append("attendance below 75")
    if a["income"] > 300000: reasons.append("income above limit")
    if reasons:
        print(a["name"], "NOT ELIGIBLE:", ", ".join(reasons))
    else:
        eligible.append(a)
eligible.sort(key=lambda a: (-a["aggregate"], -a["attendance"], a["income"]))
print("\nScholarship Merit List")
for rank, a in enumerate(eligible, 1):
    print(rank, a["name"], "Aggregate:", a["aggregate"],
          "Attendance:", a["attendance"], "Income:", a["income"])
```

Test Cases

Input / condition	Expected result
Asha: 91 aggregate, subjects ≥70, 90 attendance, ■180,000	Eligible; rank 1
Charan: aggregate 79	Rejected: aggregate below 80
Divya: subject mark 68	Rejected: subject below 70

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
NOT ELIGIBLE: Charan — aggregate below 80
NOT ELIGIBLE: Divya — subject below 70
Scholarship Merit List
1 Asha Aggregate: 91 Attendance: 90 Income: 180000
2 Bala Aggregate: 84 Attendance: 82 Income: 250000
```

Conclusion

The program applies multiple eligibility conditions, explains rejection reasons, and produces a consistent merit list with tie-break rules.

2. Progressive Electricity Billing

Problem Statement

Create a household electricity billing program that charges consumption through progressive slabs, adds a fixed charge and surcharge, applies a rebate when eligible, compares monthly bills, and flags an unusually large increase.

Objectives

• Calculate slab-wise energy charges correctly.

• Apply fixed charge, surcharge, and rebate in a clear order.

• Compare bills for multiple months and flag unusual increases.

Assumptions

• Slabs: first 100 units ■1.50/unit; next 100 ■2.50/unit; above 200 ■4/unit.

• Fixed charge ■50; surcharge 5% of energy charge; rebate ■40 if usage ≤100 units.

• An increase greater than 30% from the previous month is unusual.

Algorithm

1. Read monthly unit usage.

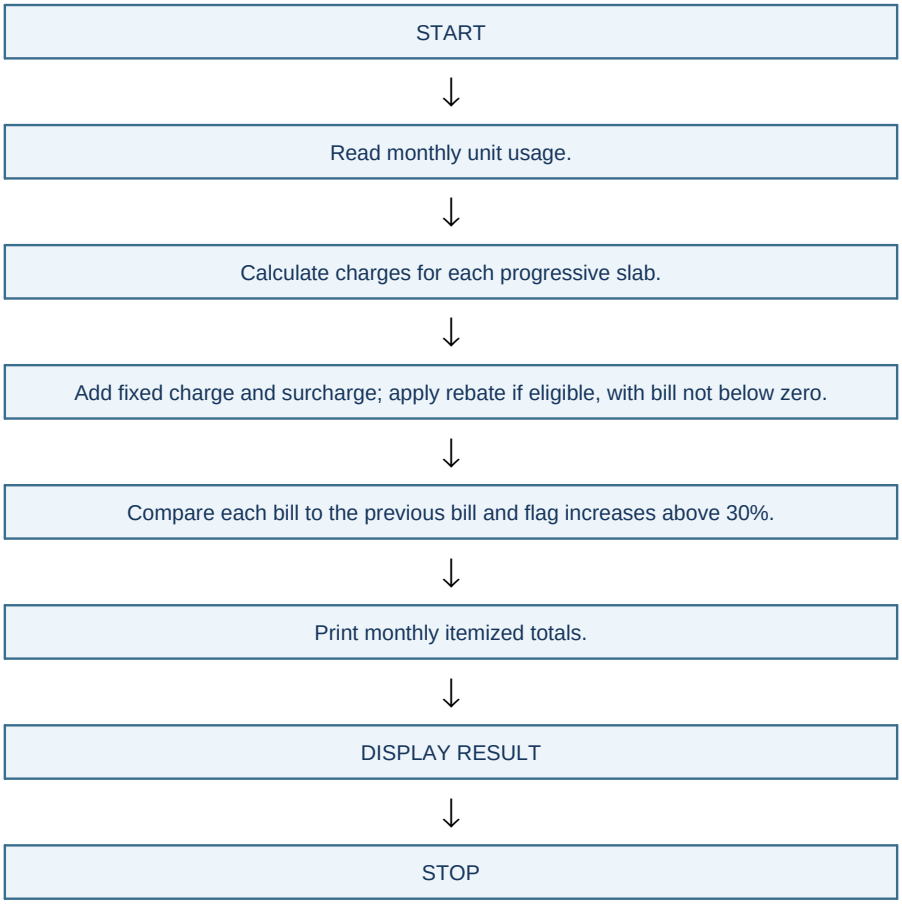
2. Calculate charges for each progressive slab.

3. Add fixed charge and surcharge; apply rebate if eligible, with bill not below zero.

4. Compare each bill to the previous bill and flag increases above 30%.

5. Print monthly itemized totals.

Flowchart



Pseudocode

```
START\nREAD monthly units\nFOR each month\n  CALCULATE slab charges progressively\n  ADD fixed charge and 5% surcharge\n  IF
```

Python Source Code

```
def energy_charge(units):
    first = min(units, 100) * 1.50
    second = min(max(units - 100, 0), 100) * 2.50
    third = max(units - 200, 0) * 4.00
    return first + second + third

monthly_units = [80, 95, 230]
previous_bill = None
for month, units in enumerate(monthly_units, 1):
    energy = energy_charge(units)
    surcharge = energy * 0.05
    bill = energy + 50 + surcharge
    rebate = 40 if units <= 100 else 0
    bill = max(0, bill - rebate)
    status = ""
    if previous_bill is not None and bill > previous_bill * 1.30:
        status = "  ** UNUSUAL INCREASE **"
    print(f"Month {month}: {units} units | Energy ■{energy:.2f} | "
          f"Bill ■{bill:.2f}{status}")
    previous_bill = bill
```

Test Cases

Input / condition	Expected result
80 units	Energy ■120.00; bill ■136.00
95 units	Energy ■142.50; bill ■159.63
230 units after 95 units	Bill increase exceeds 30%; flagged

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Month 1: 80 units | Energy ■120.00 | Bill ■136.00
Month 2: 95 units | Energy ■142.50 | Bill ■159.63
Month 3: 230 units | Energy ■470.00 | Bill ■543.50  ** UNUSUAL INCREASE **
```

Conclusion

The program calculates progressive slab charges and identifies a sudden bill increase for further checking.

3. Hospital Billing and Patient Classification

Problem Statement

Prepare a hospital bill from consultation, diagnostic, and treatment charges. Apply a concession for qualifying patients, validate charges, and classify the final payment under a supplied payment policy.

Objectives

• Calculate itemized hospital charges.

• Apply concession rules and reject invalid negative inputs.

• Classify the payable amount into a policy category.

Assumptions

• All charges must be non-negative.

• Senior citizens aged 60+ receive a 10% concession on consultation and diagnostics only.

• Final bill < ■2,000 is Basic, ■2,000–■10,000 is Standard, and above ■10,000 is High.

Algorithm

1. Read age and charge components.

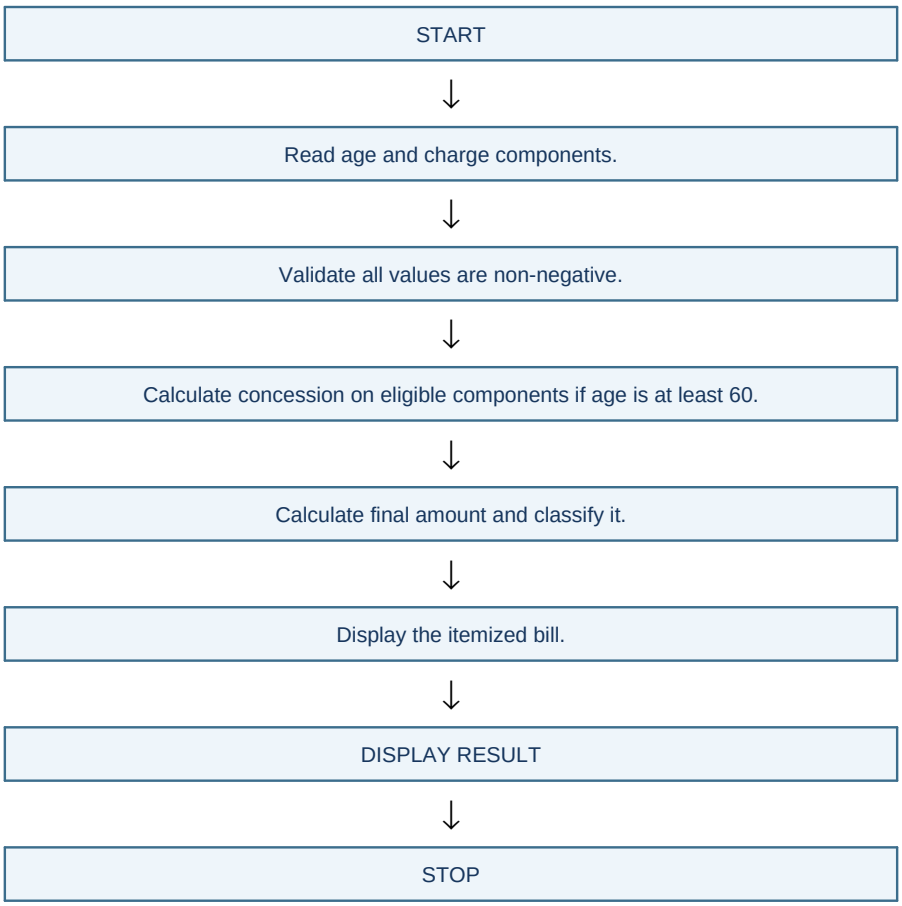
2. Validate all values are non-negative.

3. Calculate concession on eligible components if age is at least 60.

4. Calculate final amount and classify it.

5. Display the itemized bill.

Flowchart



Pseudocode

```
START\nREAD age, consultation, diagnostics, treatment\nIF any charge < 0: DISPLAY error and STOP\nIF age >= 60: concession =
```

Python Source Code

```
age = 67
consultation = 800.00
diagnostics = 2500.00
treatment = 6000.00
charges = [consultation, diagnostics, treatment]
if any(value < 0 for value in charges):
    print("Error: charges cannot be negative.")
else:
    concession = 0.10 * (consultation + diagnostics) if age >= 60 else 0
    total = sum(charges)
    payable = total - concession
    if payable < 2000:
        category = "Basic"
    elif payable <= 10000:
        category = "Standard"
    else:
        category = "High"
    print(f"Consultation: ■{consultation:.2f}")
    print(f"Diagnostics: ■{diagnostics:.2f}")
    print(f"Treatment: ■{treatment:.2f}")
    print(f"Concession: ■{concession:.2f}")
    print(f"Payable: ■{payable:.2f}")
    print("Patient billing class:", category)
```

Test Cases

Input / condition	Expected result
Age 67; charges ■800, ■2,500, ■6,000	Concession ■330; payable ■8,970; Standard
Age 30; same charges	No concession; payable ■9,300; Standard
Negative diagnostic charge	Error message

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Consultation: ■800.00
Diagnostics: ■2500.00
Treatment: ■6000.00
Concession: ■330.00
Payable: ■8970.00
Patient billing class: Standard
```

Conclusion

The solution creates an itemized bill, applies a conditional concession, and classifies the final payable amount.

4. Employee Payroll and Deduction Analysis

Problem Statement

Calculate employee gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions; and print a formatted salary statement.

Objectives

• Compute gross salary from multiple components.

• Apply progressive deductions to taxable salary.

• Show gross pay, deduction, and net pay.

Assumptions

• HRA is 20% of basic pay and other allowance is 10% of basic pay.

• Overtime is paid at ■200 per hour; incentive is supplied separately.

• Deduction: 0% on first ■10,000; 5% on next ■10,000; 10% above ■20,000 of gross salary.

Algorithm

1. Read basic pay, overtime hours, and incentive.

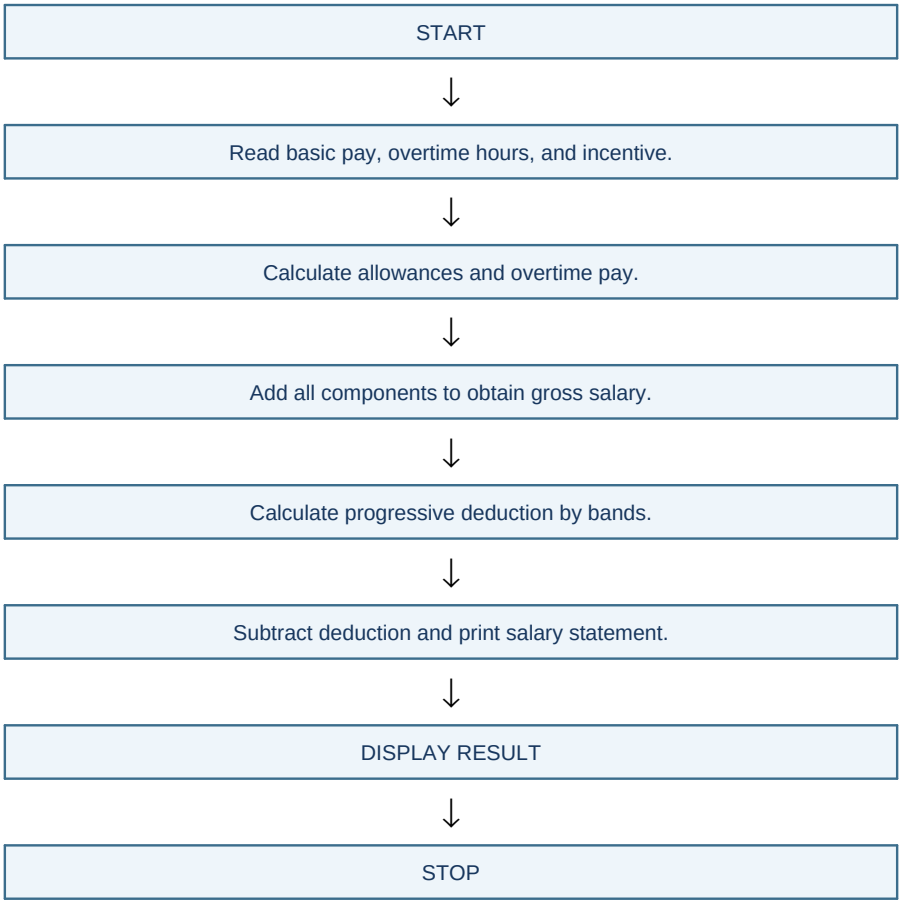
2. Calculate allowances and overtime pay.

3. Add all components to obtain gross salary.

4. Calculate progressive deduction by bands.

5. Subtract deduction and print salary statement.

Flowchart



Pseudocode

```
START\nREAD basic pay, overtime hours, incentive\nHRA = 20% of basic; other allowance = 10% of basic\nOT pay = hours * 200\n
```

Python Source Code

```
basic = 25000.00
overtime_hours = 6
incentive = 1500.00
hra = basic * 0.20
other_allowance = basic * 0.10
overtime_pay = overtime_hours * 200
gross = basic + hra + other_allowance + overtime_pay + incentive
deduction = min(gross, 10000) * 0
deduction += min(max(gross - 10000, 0), 10000) * 0.05
deduction += max(gross - 20000, 0) * 0.10
net = gross - deduction
print("SALARY STATEMENT")
print(f"Basic pay:       ■{basic:,.2f}")
print(f"HRA:             ■{hra:,.2f}")
print(f"Other allowance: ■{other_allowance:,.2f}")
print(f"Overtime:        ■{overtime_pay:,.2f}")
print(f"Incentive:       ■{incentive:,.2f}")
print(f"Gross salary:    ■{gross:,.2f}")
print(f"Deduction:       ■{deduction:,.2f}")
print(f"Net salary:      ■{net:,.2f}")
```

Test Cases

Input / condition	Expected result
Basic ■25,000, 6 overtime hours, incentive ■1,500	Gross ■35,700; deduction ■2,070; net ■33,630
Basic ■8,000, no OT or incentive	Gross ■10,400; deduction ■20; net ■10,380
Negative overtime hours	Should be rejected in a validated extension

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
SALARY STATEMENT
Basic pay:       ■25,000.00
HRA:             ■5,000.00
Other allowance: ■2,500.00
Overtime:        ■1,200.00
Incentive:       ■1,500.00
Gross salary:    ■35,200.00
Deduction:       ■2,520.00
Net salary:      ■32,680.00
```

Conclusion

The program combines salary components, applies progressive deductions, and produces a readable payroll statement.

5. Multi-Plan Telecommunications Cost Optimization

Problem Statement

Compare mobile plans against a customer's call/data usage, validity, and budget. Calculate excess usage charges and select the least expensive plan that meets all constraints.

Objectives

• Estimate total cost for each plan.

• Exclude plans that fail validity or budget requirements.

• Identify the cheapest eligible option.

Assumptions

• Usage and plan allowance are measured in GB.

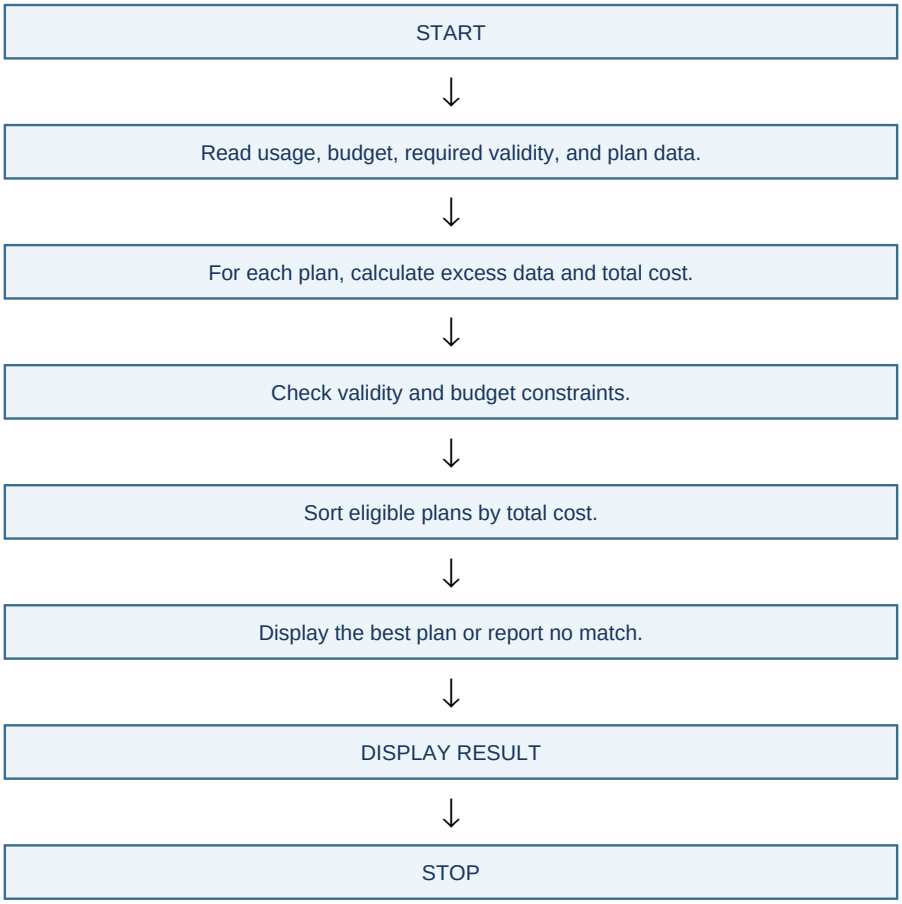
• Each plan lists price, included data, validity days, and excess cost per GB.

• A plan is eligible when validity is at least 28 days and total cost is within ■500.

Algorithm

1. Read usage, budget, required validity, and plan data.
2. For each plan, calculate excess data and total cost.
3. Check validity and budget constraints.
4. Sort eligible plans by total cost.
5. Display the best plan or report no match.

Flowchart



Pseudocode

```
START\nREAD usage, budget, required validity, plans\nFOR each plan\n  excess = max(usage - included data, 0)\n  cost = price
```

Python Source Code

```
usage_gb = 18
budget = 500
required_days = 28
plans = [
    {"name": "Basic", "price": 249, "data": 12, "days": 28, "rate": 20},
    {"name": "Smart", "price": 399, "data": 20, "days": 56, "rate": 15},
    {"name": "Max", "price": 449, "data": 30, "days": 28, "rate": 10},
]
eligible = []
for p in plans:
    excess = max(usage_gb - p["data"], 0)
    total = p["price"] + excess * p["rate"]
    print(f'{p["name"]}: total ■{total:.2f}')
    if p["days"] >= required_days and total <= budget:
        eligible.append((total, p["name"]))
if eligible:
    eligible.sort()
    print("Most economical eligible plan:", eligible[0][1],
          f"(■{eligible[0][0]:.2f})")
else:
    print("No plan meets the constraints.")
```

Test Cases

Input / condition	Expected result
18 GB; budget ■500; 28 days	Basic ■369; Smart ■399; Max ■449; Basic cheapest
40 GB usage	Excess charges are included in total
Budget ■200	No eligible plan

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Basic: total ■369.00
Smart: total ■399.00
Max: total ■449.00
Most economical eligible plan: Basic (■369.00)
```

Conclusion

The program compares usage-adjusted costs and recommends the least expensive plan that meets the stated constraints.

6. Railway Fare and Reservation Calculator

Problem Statement

Calculate fares for several passengers using distance slabs, travel class, age-based concessions, reservation fees, and cancellation penalties.

Objectives

• Calculate distance-based fare by class.

• Apply age concession and reservation fees.

• Estimate refund after a cancellation penalty.

Assumptions

• Base fare: first 100 km ■1.20/km, next 200 km ■1/km, beyond 300 km ■0.80/km.

• Class multipliers: Sleeper 1.0, AC 2.0.

• Children under 5 travel free; passengers aged 60+ receive 20% base-fare concession.

• Reservation fee is ■40 per passenger; cancellation penalty is 25% of fare before reservation fee.

Algorithm

1. Read distance, class, and passenger ages.

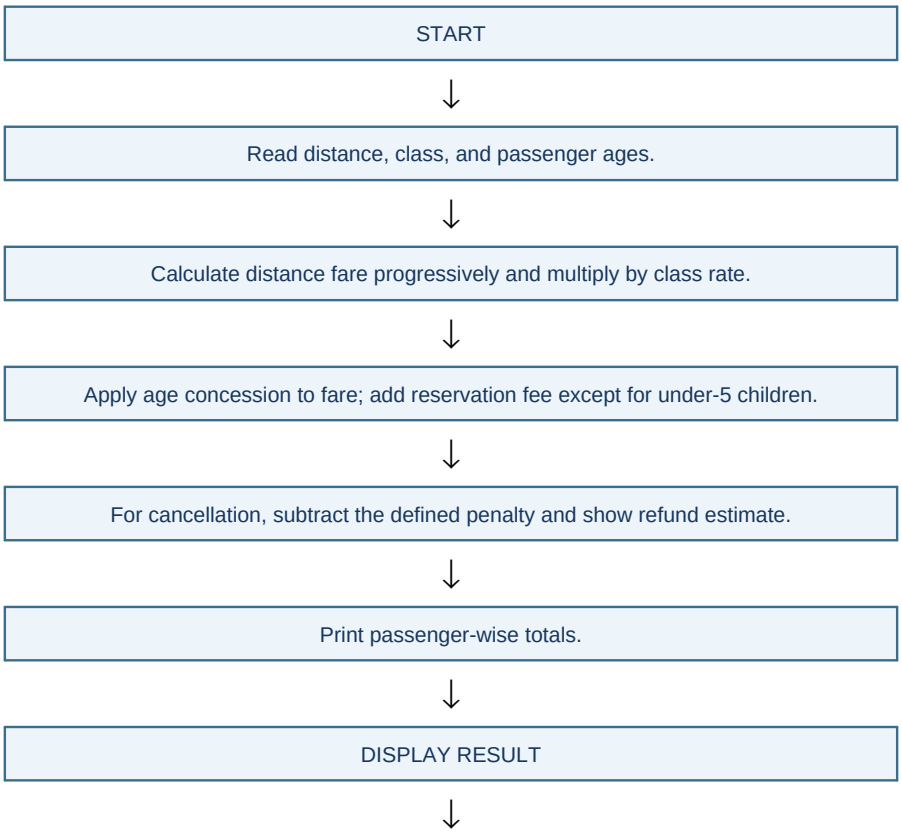
2. Calculate distance fare progressively and multiply by class rate.

3. Apply age concession to fare; add reservation fee except for under-5 children.

4. For cancellation, subtract the defined penalty and show refund estimate.

5. Print passenger-wise totals.

Flowchart



STOP

Pseudocode

```
START\nREAD distance, class, passenger ages\nCALCULATE progressive base fare\nMULTIPLY by class factor\nFOR each passenger\n
```

Python Source Code

```
distance = 350
travel_class = "AC"
passengers = [{"name": "Ravi", "age": 35}, {"name": "Meena", "age": 65},
              {"name": "Kiran", "age": 3}]
class_factor = {"Sleeper": 1.0, "AC": 2.0}[travel_class]
base = min(distance, 100) * 1.20
base += min(max(distance - 100, 0), 200) * 1.00
base += max(distance - 300, 0) * 0.80
base *= class_factor
for p in passengers:
    if p["age"] < 5:
        fare, reservation = 0.0, 0.0
    else:
        fare = base * (0.80 if p["age"] >= 60 else 1.0)
        reservation = 40.0
    total = fare + reservation
    penalty = fare * 0.25
    refund = max(0, total - penalty)
    print(f'{p["name"]} (age {p["age"]}): Fare ■{fare:.2f}, '
          f'Reservation ■{reservation:.2f}, Total ■{total:.2f}, '
          f'Cancellation refund ■{refund:.2f}')
```

Test Cases

Input / condition	Expected result
350 km AC, age 35	Fare ■700; reservation ■40; total ■740
350 km AC, age 65	Fare ■560; reservation ■40; total ■600
Age 3	Fare and reservation ■0

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Ravi (age 35): Fare ■700.00, Reservation ■40.00, Total ■740.00, Cancellation refund ■565.00
Meena (age 65): Fare ■560.00, Reservation ■40.00, Total ■600.00, Cancellation refund ■460.00
Kiran (age 3): Fare ■0.00, Reservation ■0.00, Total ■0.00, Cancellation refund ■0.00
```

Conclusion

The calculator demonstrates slab-based fares, passenger concessions, reservation fees, and a simple cancellation-refund estimate.

7. Retail Billing with Multiple Discount Rules

Problem Statement

Generate a retail invoice for multiple products with quantities, category-specific tax, threshold discounts, and a maximum concession limit.

Objectives

• Calculate line totals and category tax.

• Apply a bill-level discount when the threshold is met.

• Cap the discount and produce an itemized invoice.

Assumptions

• Clothing tax 5%; electronics tax 18%; groceries tax 0%.

• A 10% discount applies when pre-tax subtotal is at least ■5,000.

• Maximum discount is ■1,000; discount is applied before tax.

Algorithm

1. Read product, category, price, and quantity.

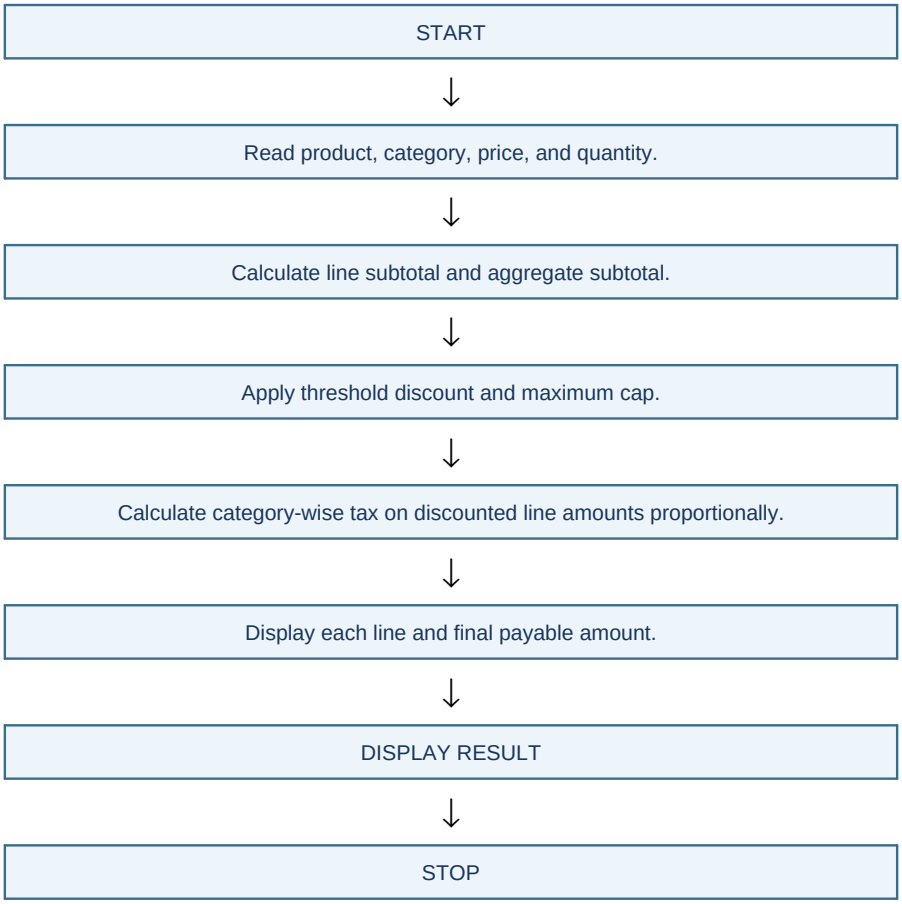
2. Calculate line subtotal and aggregate subtotal.

3. Apply threshold discount and maximum cap.

4. Calculate category-wise tax on discounted line amounts proportionally.

5. Display each line and final payable amount.

Flowchart



Pseudocode

```
START\nREAD item records\nCALCULATE each line subtotal and subtotal total\nIF subtotal >= 5000: discount = min(10% * subtota
```

Python Source Code

```
items = [
    {"name": "Jacket", "category": "clothing", "price": 2200, "qty": 1},
    {"name": "Headphones", "category": "electronics", "price": 1800, "qty": 2},
    {"name": "Rice bag", "category": "groceries", "price": 900, "qty": 1},
]
tax_rates = {"clothing": 0.05, "electronics": 0.18, "groceries": 0.00}
subtotal = sum(i["price"] * i["qty"] for i in items)
discount = min(subtotal * 0.10, 1000) if subtotal >= 5000 else 0
discount_rate = discount / subtotal if subtotal else 0
tax_total = 0
print("ITEMIZED INVOICE")
for i in items:
    line = i["price"] * i["qty"]
    discounted_line = line * (1 - discount_rate)
    tax = discounted_line * tax_rates[i["category"]]
    tax_total += tax
    print(f'{i["name"]}: {i["qty"]} x ■{i["price"]} = ■{line:.2f}; '
          f'Discount ■{line * discount_rate:.2f}; Tax ■{tax:.2f}')
print(f"Subtotal: ■{subtotal:.2f}")
print(f"Discount: ■{discount:.2f}")
print(f"Tax total: ■{tax_total:.2f}")
print(f"Amount payable: ■{subtotal - discount + tax_total:.2f}")
```

Test Cases

Input / condition	Expected result
Subtotal ■6,700	Discount capped at ■670 (10%); tax added by category
Subtotal ■4,900	No threshold discount
Subtotal ■20,000	Discount capped at ■1,000

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
ITEMIZED INVOICE
Jacket: 1 x ■2200 = ■2200.00; Discount ■220.00; Tax ■99.00
Headphones: 2 x ■1800 = ■3600.00; Discount ■360.00; Tax ■583.20
Rice bag: 1 x ■900 = ■900.00; Discount ■90.00; Tax ■0.00
Subtotal: ■6700.00
Discount: ■670.00
Tax total: ■682.20
Amount payable: ■6712.20
```

Conclusion

The solution combines product-level calculations with threshold discounts, a discount cap, and category-based taxes.

8. Manufacturing Cost and Profitability Analysis

Problem Statement

Calculate manufacturing cost from materials, wastage, labour, and machine costs, then determine whether the expected selling price reaches a target profit percentage.

Objectives

• Compute material cost including wastage.

• Add labour and machine costs to total production cost.

• Calculate profit margin and test a target.

Assumptions

• Material wastage is a percentage of raw material cost.

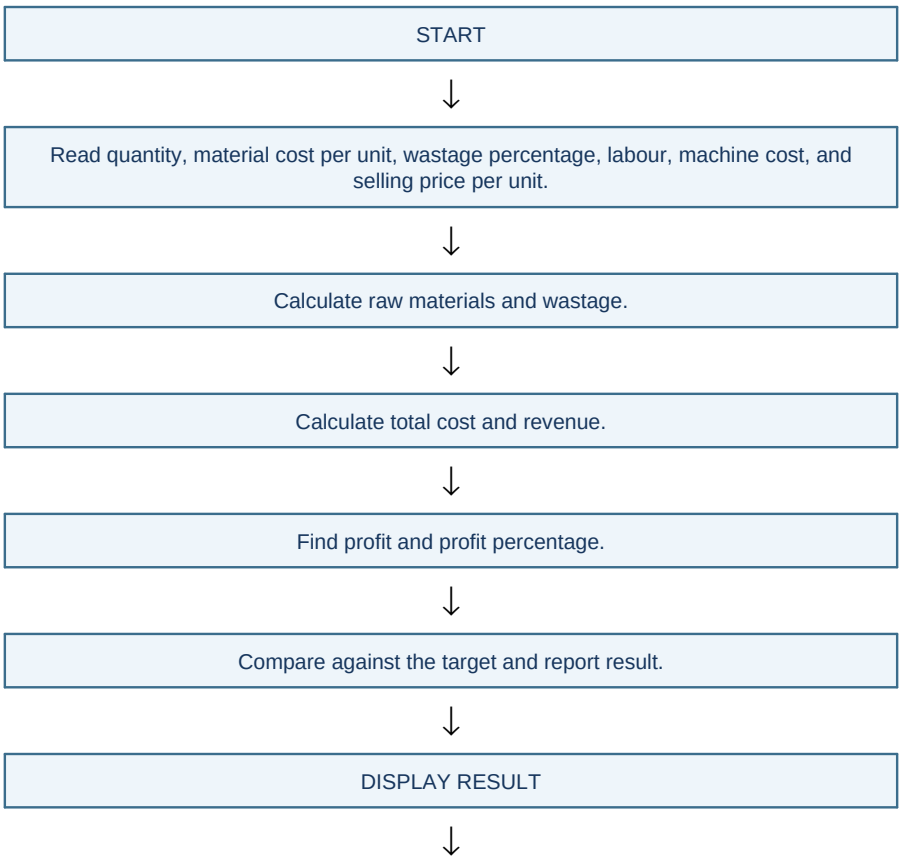
• Total production cost equals materials plus wastage, labour, and machine cost.

• Profit percentage is profit divided by total cost; target is 20%.

Algorithm

1. Read quantity, material cost per unit, wastage percentage, labour, machine cost, and selling price per unit.
2. Calculate raw materials and wastage.
3. Calculate total cost and revenue.
4. Find profit and profit percentage.
5. Compare against the target and report result.

Flowchart



STOP

Pseudocode

```
START\nREAD production quantity and costs\nmaterial = quantity * material cost per unit\nwastage = material * wastage percen
```

Python Source Code

```
quantity = 500
material_per_unit = 80.0
wastage_percent = 5
labour_cost = 12000.0
machine_cost = 8000.0
selling_price = 125.0
material = quantity * material_per_unit
wastage = material * wastage_percent / 100
total_cost = material + wastage + labour_cost + machine_cost
revenue = quantity * selling_price
profit = revenue - total_cost
profit_percent = profit / total_cost * 100 if total_cost else 0
print(f"Material cost: ■{material:.2f}")
print(f"Wastage cost: ■{wastage:.2f}")
print(f"Labour cost: ■{labour_cost:.2f}")
print(f"Machine cost: ■{machine_cost:.2f}")
print(f"Total cost: ■{total_cost:.2f}")
print(f"Revenue: ■{revenue:.2f}")
print(f"Profit: ■{profit:.2f}")
print(f"Profit on cost: {profit_percent:.2f}%")
print("Target status:", "MET" if profit_percent >= 20 else "NOT MET")
```

Test Cases

Input / condition	Expected result
500 units, ■80 material/unit, 5% wastage, ■125 selling price	Cost ■62,000; revenue ■62,500; profit ■500; target not met
Increase selling price to ■150	Recalculate; target may be met
Quantity 0	Handle division safely; profit percentage set to 0

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Material cost: ■40000.00
Wastage cost: ■2000.00
Labour cost: ■12000.00
Machine cost: ■8000.00
Total cost: ■62000.00
Revenue: ■62500.00
Profit: ■500.00
Profit on cost: 0.81%
Target status: NOT MET
```

Conclusion

The program quantifies production costs and checks profitability against a defined target.

9. Multi-Currency Travel Budget Analysis

Problem Statement

Convert travel expenses in multiple currencies using supplied exchange rates, include conversion fees, compare total spending with a budget, and identify the largest expense category.

Objectives

• Convert foreign-currency costs to home currency.

• Include conversion fees in the total.

• Compare total spending with the budget and identify the largest category.

Assumptions

• All exchange rates are supplied as INR per one foreign-currency unit.

• A 2% conversion fee applies to converted expense amounts.

• The budget is in INR; fees are calculated after conversion.

Algorithm

1. Read expenses, currencies, and exchange rates.

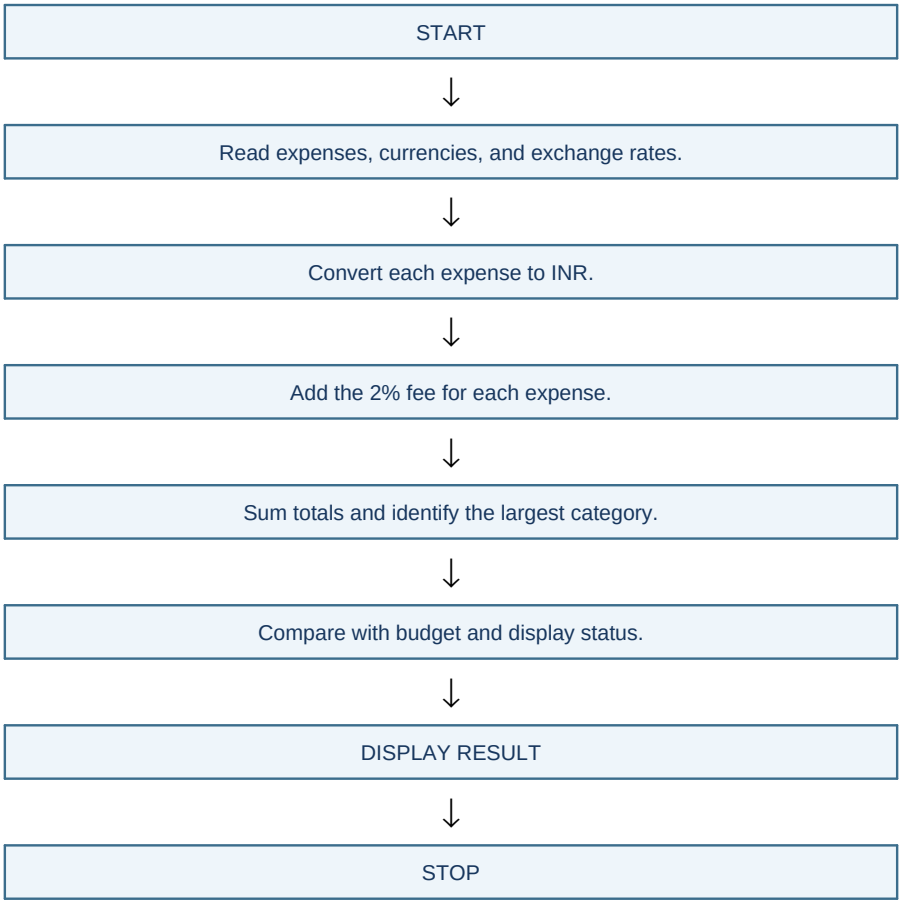
2. Convert each expense to INR.

3. Add the 2% fee for each expense.

4. Sum totals and identify the largest category.

5. Compare with budget and display status.

Flowchart



Pseudocode

```
START\nREAD expenses, rates, budget\nFOR each expense\n  converted = amount * rate\n  fee = converted * 2%\n  total category
```

Python Source Code

```
rates = {"USD": 83.0, "EUR": 90.0, "INR": 1.0}
expenses = [
    {"category": "Hotel", "amount": 200, "currency": "USD"},
    {"category": "Food", "amount": 80, "currency": "EUR"},
    {"category": "Local transport", "amount": 3000, "currency": "INR"},
]
budget = 30000.0
totals = {}
grand_total = 0
for e in expenses:
    converted = e["amount"] * rates[e["currency"]]
    fee = converted * 0.02
    total = converted + fee
    totals[e["category"]] = total
    grand_total += total
    print(f'{e["category"]}: converted ■{converted:.2f}, '
          f'fee ■{fee:.2f}, total ■{total:.2f}')
largest = max(totals, key=totals.get)
print(f"Total spending: ■{grand_total:.2f}")
print("Largest expense category:", largest)
print("Budget status:", "WITHIN BUDGET" if grand_total <= budget else "OVER BUDGET")
```

Test Cases

Input / condition	Expected result
Hotel USD 200 at ■83/USD	Converted ■16,600; fee ■332; total ■16,932
Budget ■30,000	Total is within budget for sample expenses
Missing exchange rate	Should be validated and reported before calculation

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Hotel: converted ■16600.00, fee ■332.00, total ■16932.00
Food: converted ■7200.00, fee ■144.00, total ■7344.00
Local transport: converted ■3000.00, fee ■60.00, total ■3060.00
Total spending: ■27336.00
Largest expense category: Hotel
Budget status: WITHIN BUDGET
```

Conclusion

The analysis consolidates expenses into INR, accounts for conversion fees, and identifies the biggest expense category.

10. Campus Event Financial Planning

Problem Statement

Estimate campus event expenses for venue, equipment, catering, transport, and certificates. Calculate per-participant contribution, sponsorship shortfall, and minimum registrations needed to break even.

Objectives

• Calculate total fixed and variable event costs.

• Determine net funding required after sponsorship.

• Calculate participant contribution and break-even registration count.

Assumptions

• Venue, equipment, and transport are fixed costs; catering and certificates are per participant.

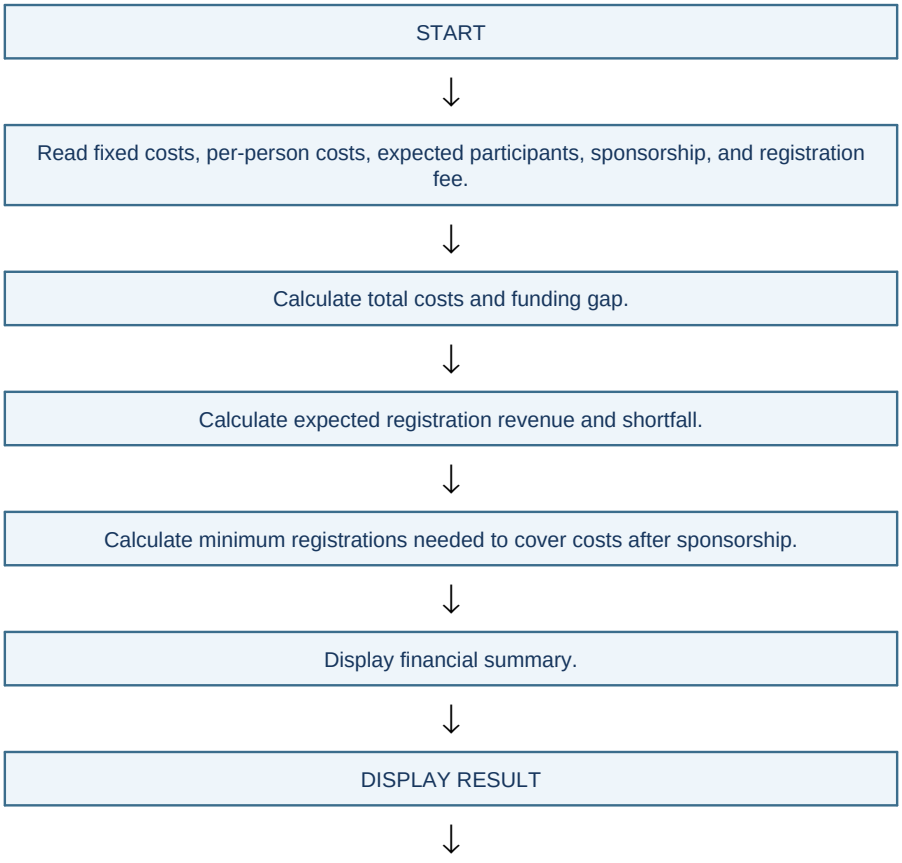
• Sponsorship reduces the total amount to be funded.

• Registration contribution is ■500 per participant.

Algorithm

1. Read fixed costs, per-person costs, expected participants, sponsorship, and registration fee.
2. Calculate total costs and funding gap.
3. Calculate expected registration revenue and shortfall.
4. Calculate minimum registrations needed to cover costs after sponsorship.
5. Display financial summary.

Flowchart



STOP

Pseudocode

```
START\nREAD fixed costs, per-person costs, participant count, sponsorship, fee\nfixed = venue + equipment + transport\nvaria
```

Python Source Code

```
import math
venue = 12000.0
equipment = 5000.0
transport = 3000.0
catering_per_person = 180.0
certificate_per_person = 20.0
participants = 150
sponsorship = 15000.0
registration_fee = 500.0
fixed_cost = venue + equipment + transport
variable_per_person = catering_per_person + certificate_per_person
total_cost = fixed_cost + variable_per_person * participants
registration_revenue = registration_fee * participants
funding_gap = max(total_cost - sponsorship - registration_revenue, 0)
contribution_margin = registration_fee - variable_per_person
remaining_fixed = max(fixed_cost - sponsorship, 0)
break_even = (math.ceil(remaining_fixed / contribution_margin)
              if contribution_margin > 0 else None)
print(f"Fixed costs: ■{fixed_cost:.2f}")
print(f"Variable cost per participant: ■{variable_per_person:.2f}")
print(f"Total event cost: ■{total_cost:.2f}")
print(f"Sponsorship: ■{sponsorship:.2f}")
print(f"Registration revenue: ■{registration_revenue:.2f}")
print(f"Funding shortfall: ■{funding_gap:.2f}")
print(f"Minimum registrations to break even: {break_even}")
```

Test Cases

Input / condition	Expected result
150 participants; ■15,000 sponsorship; ■500 fee	Total cost ■51,000; registration revenue ■75,000; no shortfall
Registration fee ■200; variable cost ■200	No positive contribution margin; break-even impossible without more funding
Zero participants	Total cost equals fixed costs

Sample Output Screen

Illustrative console output (run the code to capture an actual screenshot):

```
Fixed costs: ■20000.00
Variable cost per participant: ■200.00
Total event cost: ■50000.00
Sponsorship: ■15000.00
Registration revenue: ■75000.00
Funding shortfall: ■0.00
Minimum registrations to break even: 15
```

Conclusion

The program supports event budgeting by separating fixed and variable costs, tracking funding sources, and estimating break-even registrations.